

Sequence-to-Sequence Models

1. Overview

Sequence-to-sequence (seq2seq) models map an input sequence to an output sequence, often with different lengths. They are used in machine translation, summarization, question answering, dialogue, speech recognition, code generation, and structured prediction.

2. Intuition

Think of seq2seq as a reader-writer system. The encoder reads the input sentence and builds a representation. The decoder writes the answer token by token, using what the encoder understood.

Example: English input "I love NLP" -> Hindi output "मुझे NLP पसंद है".

3. Prerequisites

- Tokenization, embeddings, softmax, cross-entropy
- RNN/LSTM/GRU or Transformer encoder-decoder
- Conditional probability and maximum likelihood
- Teacher forcing and autoregressive decoding

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Encoder	Converts input tokens into hidden states	Captures source meaning	Source sentence states	Explain bottleneck in vanilla seq2seq
Decoder	Generates output tokens one at a time	Produces target sequence	Translation tokens	Why decoding is autoregressive
Attention	Learns which source tokens to focus on	Solves fixed-vector bottleneck	Focus on "cat" while generating "chat"	Derive attention weights
Teacher forcing	Feeds true previous token during training	Stabilizes learning	Input previous gold word	Exposure bias
Autoregression	Each token depends on previous generated tokens	Enables variable length output	$p(y_t$	$y_{<t}, x)$

5. Algorithm / Working Process

1. Tokenize source and target text.
2. Encoder produces contextual states $h_1, \dots, h_n$.
3. Decoder starts with $\langle \text{bos} \rangle$.
4. At each step, decoder attends to encoder states and predicts next-token distribution.

5. Training uses gold previous tokens and cross-entropy.
6. Inference uses greedy decoding, beam search, or sampling until `<eos>`.

6. Mathematical Foundation

The model estimates:

$$P(y \mid x) = \prod_t P(y_t \mid y_1, \dots, y_{t-1}, x)$$

Training minimizes negative log-likelihood:

$$L = - \sum_t \log P(y_t^* \mid y_{<t}^*, x)$$

Attention:

$$\text{score}_{ti} = q_t^T k_i / \sqrt{d_k}$$

$$\alpha_{ti} = \text{softmax}(\text{score}_{ti})$$

$$\text{context}_t = \sum_i \alpha_{ti} v_i$$

7. Practical Implementation

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

model_name = "t5-small"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

text = "translate English to German: Sequence models are useful."
inputs = tokenizer(text, return_tensors="pt")

outputs = model.generate(
    **inputs,
    max_new_tokens=40,
    num_beams=4,
    early_stopping=True,
)

print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

8. Code Explanation

- `AutoTokenizer` converts text into model token IDs.
- `AutoModelForSeq2SeqLM` loads an encoder-decoder language model.
- The task prefix tells T5 what transformation to perform.
- `generate` performs autoregressive decoding.
- `num_beams=4` keeps four candidate outputs during decoding.

9. Training / Evaluation

- Use paired source-target examples.
- Split by document/user/time where leakage is possible.
- Common metrics: BLEU, ROUGE, METEOR, chrF, exact match, human evaluation.
- Improve with better tokenization, larger pretrained models, data cleaning, beam tuning, LoRA fine-tuning, and task-specific prompts.

10. Complexity and Cost

- Transformer attention cost is roughly $O(n^2 d)$ per layer for sequence length n .
- Decoder generation is slower than classification because tokens are produced sequentially.
- Memory grows with sequence length, layers, hidden size, and beam width.

11. Common Use Cases

- Translation, summarization, text simplification, grammar correction, dialogue, semantic parsing, speech-to-text, code translation.

12. Common Mistakes

- Training with target leakage.
- Comparing generated text using accuracy only.
- Ignoring exposure bias.
- Setting beam width too high and getting dull outputs.
- Not handling max length and truncation carefully.

13. Edge Cases / Limitations

- Long documents exceed context windows.
- Rare names and numbers may be copied incorrectly.
- Hallucination in open-ended generation.
- Error accumulation during decoding.

14. Variations

- RNN encoder-decoder: useful for learning basics, less common in production.
- Attention-based seq2seq: important historically and conceptually.
- Transformer encoder-decoder: standard for translation and summarization.
- Pointer-generator: useful when copying source tokens matters.
- Multimodal seq2seq: image captioning and speech tasks.

15. Related Topics

- RNN vs Transformer: recurrence vs parallel self-attention.
- Fine-tuning vs prompting: update weights vs condition pretrained model.
- Seq2seq vs decoder-only LLM: encoder-decoder explicitly conditions on input; decoder-only concatenates prompt and output.

16. Interview Questions

1. What is a seq2seq model? It maps one sequence to another, often using encoder-decoder architecture.
2. Why is attention useful? It lets the decoder focus on relevant source positions instead of relying on one fixed vector.
3. What is teacher forcing? Feeding the true previous output token during training.
4. What is exposure bias? The model trains on gold prefixes but infers from its own generated prefixes.
5. Why is decoding slower than classification? It generates one token at a time.
6. What is the seq2seq loss? Sum of token-level cross-entropy losses.
7. How do you evaluate translation? BLEU, chrF, COMET, and human judgment.
8. Why can seq2seq hallucinate? It optimizes likely output, not guaranteed factual grounding.
9. What happens with long inputs? Attention cost and truncation become problems.
10. Encoder-decoder vs decoder-only? Encoder-decoder separates input encoding and output generation; decoder-only predicts continuations.

17. Practice Tasks

- Code task: run T5 for translation and compare greedy vs beam output.
- Dataset project: fine-tune on a small summarization dataset.
- Experiment: vary beam width and max length.
- Debugging: inspect outputs for repetition and truncation.
- Extension: add ROUGE evaluation.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Meeting Summarizer	Converts transcripts to summaries	HF Transformers, Streamlit	SAMSum	Practical NLP product
Text Simplifier	Rewrites complex text simply	T5, PyTorch	WikiLarge	Education use case
Translation Demo	Domain-specific translation	MarianMT/T5	OPUS	Classic seq2seq project

19. Quick Revision

- Key idea: encode input, decode output token by token.
- Main formula: $P(y|x) = \prod_t P(y_t | y_{<t}, x)$.
- Metrics: BLEU, ROUGE, exact match, human evaluation.
- Trap: training/inference mismatch.
- Interview one-liner: Seq2seq learns conditional generation from one sequence to another.

20. Final Cheat Sheet

Item	Notes
Definition	Model that maps input sequence to output sequence
Input/output	Tokens in, generated tokens out

Item	Notes
Steps	Encode, attend, decode, stop
Hyperparameters	max length, beam width, learning rate
Pros	Flexible, strong for generation
Cons	Slow decoding, hallucination, length issues
Best use	Translation, summarization, QA generation

Beam Search

1. Overview

Beam search is an approximate decoding algorithm for sequence generation. Instead of keeping only the best next token, it keeps the top k partial sequences, called beams. It is widely used in machine translation, summarization, speech recognition, OCR, and seq2seq inference.

2. Intuition

Greedy decoding is like choosing the best word at every step without thinking ahead. Beam search keeps several promising sentences alive, so a slightly worse first word can still win if it leads to a better full sentence.

3. Prerequisites

- Language-model probabilities
- Log probabilities
- Autoregressive generation
- Search algorithms

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Beam width	Number of candidates kept	Controls quality/cost tradeoff	$k=4$	Larger is not always better
Log score	Sum of log token probabilities	Avoids underflow	$\sum \log p_t$	Why not multiply probabilities
Length penalty	Normalizes for sequence length	Avoids short-output bias	divide by length factor	Explain brevity problem
Early stopping	Stops when beams finish	Saves compute	all beams hit $\langle \text{eos} \rangle$	Correct stopping condition
Diversity	Avoids near-duplicate beams	Useful for creative outputs	diverse beam search	Beam vs sampling

5. Algorithm / Working Process

1. Start with sequence `<bos>` and score `0`.
2. Expand each active beam with all possible next tokens.
3. Add log probability to each candidate score.
4. Keep top `k` candidates.
5. Move completed sequences ending in `<eos>` to finished list.
6. Return the highest scoring finished sequence, often with length penalty.

6. Mathematical Foundation

Candidate score:

$$\text{score}(y_{1:t}) = \sum_{i=1}^t \log P(y_i \mid y_{<i}, x)$$

Length-normalized score:

$$\text{score_norm} = \text{score} / ((5 + \text{len}(y)) / 6)^\alpha$$

where `alpha` controls how much longer sequences are favored.

7. Practical Implementation

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

tokenizer = AutoTokenizer.from_pretrained("t5-small")
model = AutoModelForSeq2SeqLM.from_pretrained("t5-small")

text = "summarize: Beam search keeps several likely partial outputs during decoding."
inputs = tokenizer(text, return_tensors="pt")

for beams in [1, 4, 8]:
    output = model.generate(
        **inputs,
        num_beams=beams,
        max_new_tokens=30,
        length_penalty=1.0,
        early_stopping=True,
    )
    print(beams, tokenizer.decode(output[0], skip_special_tokens=True))
```

8. Code Explanation

- `num_beams=1` is greedy decoding.
- Larger `num_beams` explores more candidates.
- `length_penalty` changes preference for short or long outputs.
- The same model can produce different outputs only by changing decoding settings.

9. Training / Evaluation

Beam search is an inference algorithm, not a training method. Evaluate generated text using task metrics and human inspection. Tune beam width, length penalty, no-repeat n-gram size, and max length on validation data.

10. Complexity and Cost

- Greedy decoding: $O(TV)$ scoring per sequence length T and vocabulary V .
- Beam search: approximately $O(kTV)$.
- Memory increases with beam width and cached decoder states.

11. Common Use Cases

- Machine translation, summarization, captioning, ASR decoding, grammar correction.

12. Common Mistakes

- Assuming larger beam always improves quality.
- Forgetting length normalization.
- Using beam search for creative generation where sampling may be better.
- Comparing decoding strategies without fixed prompts and seeds.

13. Edge Cases / Limitations

- Can produce generic, high-probability text.
- Beams may collapse into similar candidates.
- High beam width may worsen open-ended generation.
- Expensive for long outputs.

14. Variations

- Greedy search: fastest, lowest exploration.
- Diverse beam search: penalizes similar beams.
- Constrained beam search: forces required words or formats.
- Top-k/top-p sampling: better for creative outputs.

15. Related Topics

- Beam search vs sampling: optimization vs diversity.
- Decoding vs training: beam search does not change model weights.
- Seq2seq inference: beam search is a common decoder for seq2seq.

16. Interview Questions

1. What is beam search? A heuristic search that keeps top k partial sequences.
2. Why use log probabilities? To avoid numerical underflow and convert products to sums.
3. What is beam width? Number of candidates retained at every step.
4. What is the downside of large beam width? More compute and often generic outputs.
5. Why use length penalty? Raw log probability favors shorter sequences.
6. Is beam search exact? No, it is approximate.

7. Beam search vs greedy? Greedy keeps one path; beam keeps many.
8. Beam search vs sampling? Beam seeks likely output; sampling adds randomness.
9. Does beam search train the model? No, it only affects inference.
10. Where is constrained beam useful? Structured generation and required terminology.

17. Practice Tasks

- Implement toy beam search over a small vocabulary.
- Compare greedy, beam, and top-p outputs.
- Tune length penalty on summarization.
- Debug repeated phrases with `no_repeat_ngram_size`.
- Add constraints to force a keyword.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Decoding Visualizer	Shows beam expansion step by step	Python, Streamlit	Toy LM	Great interview demo
Translation Tuner	Tests decoding configs	Transformers	OPUS	Practical evaluation skill
ASR Decoder Demo	Applies beam decoding to transcripts	PyTorch	LibriSpeech subset	Shows search knowledge

19. Quick Revision

- Key idea: keep `k` best partial generations.
- Formula: sum token log probabilities.
- Use when: deterministic high-quality generation is needed.
- Trap: too-large beam may reduce diversity.
- Interview one-liner: Beam search trades compute for better sequence-level decoding than greedy.

20. Final Cheat Sheet

Item	Notes
Definition	Approximate sequence decoding search
Input/output	Token probabilities -> best sequence
Steps	Expand, score, prune, stop
Hyperparameters	beam width, length penalty, max length
Metrics	Task metric plus human quality
Pros	Better than greedy for many tasks
Cons	Slower, less diverse

Named Entity Recognition

1. Overview

Named Entity Recognition (NER) identifies spans of text that refer to entities such as people, organizations, locations, dates, products, diseases, and legal clauses. It is used in search, compliance, medical NLP, resume parsing, knowledge graph construction, and document automation.

2. Intuition

NER is like highlighting important names in a document and labeling their type. In "Satya Nadella leads Microsoft", "Satya Nadella" is PERSON and "Microsoft" is ORG.

3. Prerequisites

- Tokenization and subword tokenization
- Sequence labeling
- BIO/BIOES tagging
- Precision, recall, F1
- Transformers or CRFs

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Entity span	Continuous text segment	Output is span-level, not just token-level	New York	Token vs span metrics
Entity type	Category label	Drives downstream extraction	ORG, LOC	Domain-specific labels
BIO tags	Begin/Inside/Outside labels	Handles multi-token entities	B-LOC I-LOC	Invalid tag transitions
Contextual embeddings	Token meaning depends on sentence	Resolves ambiguity	Apple company vs fruit	Why BERT helps
CRF layer	Structured decoder over tags	Enforces valid label sequences	no I-PER after O	When CRF helps

5. Algorithm / Working Process

1. Define entity schema.
2. Annotate text using BIO tags.
3. Tokenize text and align word labels to subword tokens.
4. Feed tokens to a model such as BERT.
5. Predict label distribution for each token.

6. Decode labels into entity spans.
7. Evaluate using entity-level precision, recall, and F1.

6. Mathematical Foundation

Token classification loss:

$$L = - \sum_t \log P(y_t | x)$$

Softmax token probability:

$$P(y_t=c|x) = \exp(z_{tc}) / \sum_j \exp(z_{tj})$$

Entity-level metrics:

$$\text{Precision} = \text{correct predicted entities} / \text{predicted entities}$$

$$\text{Recall} = \text{correct predicted entities} / \text{gold entities}$$

$$F1 = 2PR / (P + R)$$

7. Practical Implementation

```
from transformers import pipeline

ner = pipeline(
    "token-classification",
    model="dslim/bert-base-NER",
    aggregation_strategy="simple",
)

text = "Sundar Pichai announced a new Google product in California."
for entity in ner(text):
    print(entity["word"], entity["entity_group"], round(entity["score"], 3))
```

8. Code Explanation

- `pipeline("token-classification")` loads a pretrained NER model.
- `aggregation_strategy="simple"` merges subword predictions into spans.
- Each output contains entity text, label, score, and character offsets.

9. Training / Evaluation

- Use labeled datasets such as CoNLL-2003, OntoNotes, or domain-specific annotations.
- Preserve document/source splits to avoid leakage.
- Metrics should be entity-level F1, not only token accuracy.
- Improve with annotation cleanup, domain adaptation, gazetteers, CRF decoding, and active learning.

10. Complexity and Cost

- Transformer NER costs $O(n^2 d)$ per layer.
- Inference is parallel across tokens and faster than generation.
- Long documents need chunking with overlap.

11. Common Use Cases

- Resume parsing, legal extraction, clinical entity extraction, news analytics, customer support routing, knowledge graph population.

12. Common Mistakes

- Reporting token accuracy when most tokens are 0.
- Misaligning labels after subword tokenization.
- Not defining annotation guidelines.
- Ignoring nested entities.
- Training on noisy weak labels without validation.

13. Edge Cases / Limitations

- Nested entities: "University of California" as both org and location phrase.
- Ambiguous names: "Amazon" river vs company.
- Emerging entities unseen in training.
- Long documents and tables.

14. Variations

- Rule-based NER: useful for simple IDs and regex-like entities.
- BiLSTM-CRF: classic sequence labeling baseline.
- BERT token classification: standard placement answer.
- Span classification: predicts start/end spans directly.
- LLM extraction: flexible but needs validation for production.

15. Related Topics

- NER vs POS tagging: semantic spans vs grammatical categories.
- NER vs relation extraction: entities first, relations between them next.
- Token classification vs text classification: per-token labels vs one label per text.

16. Interview Questions

1. What is NER? Detecting and classifying entity spans in text.
2. What is BIO tagging? A scheme marking beginning, inside, and outside of entities.
3. Why is token accuracy misleading? Most tokens may be outside entities.
4. How do subwords affect NER? Word labels must align to subword tokens.
5. What metric is preferred? Entity-level F1.
6. How does CRF help? It models label transition constraints.
7. Rule-based vs ML NER? Rules are precise for patterns; ML generalizes with context.
8. How handle domain-specific entities? Annotate domain data and fine-tune.
9. What are nested entities? Entity spans inside larger entity spans.

10. Why does BERT help NER? Contextual embeddings disambiguate token meaning.

17. Practice Tasks

- Run pretrained NER on news articles.
- Fine-tune BERT on CoNLL-2003.
- Build regex extraction for emails and compare with NER.
- Analyze false positives by entity type.
- Add overlap chunking for long documents.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Resume Entity Extractor	Extracts skills, companies, degrees	spaCy, HF	Custom resumes	Placement-relevant
Medical NER	Finds diseases and drugs	BioBERT	BC5CDR	Domain NLP
Legal Clause Extractor	Extracts dates, parties, obligations	Transformers	CUAD	AI engineer use case

19. Quick Revision

- Key idea: label entity spans.
- Formula: token cross-entropy and entity F1.
- Metrics: precision, recall, F1.
- Trap: subword-label mismatch.
- Interview one-liner: NER is sequence labeling for named spans.

20. Final Cheat Sheet

Item	Notes
Definition	Entity span detection and classification
Input/output	Text -> labeled spans
Steps	Tag, tokenize, predict, merge spans
Hyperparameters	max length, LR, label scheme
Metrics	Entity precision/recall/F1
Pros	Structured extraction from text
Cons	Annotation-heavy, domain-sensitive

Text Classification

1. Overview

Text classification assigns one or more labels to a text. It powers spam detection, intent classification, topic tagging, toxicity detection, ticket routing, sentiment analysis, and document categorization.

2. Intuition

It is like sorting emails into folders. The model reads text and predicts the most likely category.

3. Prerequisites

- Bag-of-words, TF-IDF, embeddings
- Logistic regression, naive Bayes, transformers
- Cross-entropy, sigmoid, softmax
- Train/validation/test split

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Binary classification	Two labels	Common baseline	spam/not spam	Threshold tuning
Multi-class	One label from many	Topic classification	sports/politics/tech	Softmax
Multi-label	Multiple labels allowed	Tags	finance + legal	Sigmoid per label
Feature extraction	Convert text to numbers	Model requirement	TF-IDF	Sparse vs dense
Class imbalance	Unequal label counts	Biases predictions	1% fraud	F1/PR-AUC

5. Algorithm / Working Process

1. Collect labeled texts.
2. Clean and split data.
3. Convert text to features using TF-IDF or token embeddings.
4. Train classifier.
5. Tune thresholds and hyperparameters on validation set.
6. Evaluate on held-out test set.

6. Mathematical Foundation

Binary logistic regression:

$$p = \text{sigmoid}(w^T x + b) = 1 / (1 + e^{-(w^T x + b)})$$

Binary cross-entropy:

$$L = -[y \log p + (1-y) \log(1-p)]$$

Multi-class softmax:

$$P(y=c|x) = \exp(z_c) / \sum_j \exp(z_j)$$

7. Practical Implementation

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report
from sklearn.pipeline import make_pipeline

train = fetch_20newsgroups(subset="train", categories=["sci.space", "rec.autos"])
test = fetch_20newsgroups(subset="test", categories=["sci.space", "rec.autos"])

model = make_pipeline(
    TfidfVectorizer(stop_words="english", max_features=20000),
    LogisticRegression(max_iter=1000),
)

model.fit(train.data, train.target)
pred = model.predict(test.data)
print(classification_report(test.target, pred, target_names=test.target_names))
```

8. Code Explanation

- `TfidfVectorizer` turns documents into weighted word features.
- `LogisticRegression` learns class weights.
- `make_pipeline` prevents preprocessing leakage by fitting vectorizer only on training data.
- `classification_report` shows precision, recall, and F1.

9. Training / Evaluation

- Use stratified splits when class distribution matters.
- Metrics: accuracy for balanced data, macro F1 for imbalance, ROC-AUC/PR-AUC for binary ranking.
- Improve with cleaning, n-grams, class weights, transformer fine-tuning, and threshold tuning.

10. Complexity and Cost

- TF-IDF + linear model is CPU-friendly and fast.
- Transformer classifiers need GPU for efficient training.
- Inference for linear models is sparse matrix multiplication; very cheap.

11. Common Use Cases

- Spam filtering, support ticket routing, intent detection, news categorization, moderation, document triage.

12. Common Mistakes

- Fitting vectorizer before train/test split.
- Using accuracy on imbalanced data.
- Removing negation words blindly.
- Evaluating on duplicate or near-duplicate texts.
- Ignoring threshold calibration.

13. Edge Cases / Limitations

- Sarcasm and implicit meaning.
- Domain shift.
- Very short texts.
- Out-of-vocabulary slang.
- Labels with overlapping definitions.

14. Variations

- Naive Bayes: strong fast baseline.
- SVM: good for sparse TF-IDF.
- CNN/RNN classifiers: older neural approaches.
- BERT fine-tuning: strong contextual classifier.
- Zero-shot classification: useful with no labels, less reliable.

15. Related Topics

- Text classification vs sentiment analysis: sentiment is a specific classification task.
- Multi-class vs multi-label: one label vs many labels.
- TF-IDF vs embeddings: lexical matching vs semantic representation.

16. Interview Questions

1. What is text classification? Assigning labels to text.
2. TF-IDF meaning? Term importance adjusted by corpus frequency.
3. Why use macro F1? It treats all classes equally.
4. Multi-class vs multi-label? Softmax one class vs sigmoid independent labels.
5. What is data leakage? Test information entering training/preprocessing.
6. Why logistic regression works well for text? High-dimensional sparse features are often linearly separable.
7. How handle imbalance? Class weights, resampling, threshold tuning, F1/PR-AUC.
8. When use BERT? When context and semantics matter and compute is available.
9. What is calibration? Making predicted probabilities reflect true likelihood.
10. How debug classifier errors? Inspect confusion matrix and misclassified examples.

17. Practice Tasks

- Build TF-IDF logistic classifier.
- Compare unigrams vs bigrams.
- Tune class weights on imbalanced data.
- Fine-tune DistilBERT.

- Build a confusion-matrix error report.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Support Ticket Router	Routes tickets to teams	sklearn/FastAPI	Kaggle support data	Production-style
Toxic Comment Classifier	Flags harmful text	BERT	Jigsaw	Moderation use case
Job Post Classifier	Categorizes job descriptions	TF-IDF	scraped/open data	Placement relevant

19. Quick Revision

- Key idea: text -> label.
- Formula: cross-entropy.
- Metrics: accuracy, F1, ROC-AUC, PR-AUC.
- Trap: leakage through preprocessing.
- Interview one-liner: Text classification converts unstructured text into decision labels.

20. Final Cheat Sheet

Item	Notes
Definition	Label prediction for text
Input/output	Document -> class/classes
Steps	Split, vectorize, train, evaluate
Hyperparameters	n-grams, max features, LR, threshold
Pros	Simple, useful, measurable
Cons	Domain shift, ambiguity

Sentiment Analysis

1. Overview

Sentiment analysis detects opinion polarity, emotion, or attitude in text. It is used for product reviews, social media monitoring, customer feedback, finance signals, and brand analytics.

2. Intuition

The model estimates whether a text sounds positive, negative, neutral, or emotionally specific. "The camera is amazing but battery is poor" may need aspect-level sentiment.

3. Prerequisites

- Text classification
- Polarity labels and ordinal labels
- Negation, sarcasm, and context
- F1, confusion matrix, calibration

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Polarity	Positive/negative/neutral	Basic output	5-star review	Class imbalance
Emotion	Joy/anger/sadness labels	More granular	angry tweet	Multi-label setup
Aspect sentiment	Sentiment about target aspect	Handles mixed opinions	food good, service bad	Span + classification
Negation	Words reverse meaning	Avoids false positives	"not good"	Preprocessing trap
Sarcasm	Literal words hide true sentiment	Hard for models	"Great, another delay"	Limitation

5. Algorithm / Working Process

1. Define label scheme.
2. Collect reviews/tweets/feedback.
3. Split data carefully by product/user/time.
4. Train classifier using TF-IDF or transformer.
5. Evaluate per class.
6. Inspect confusing cases such as neutral vs mixed sentiment.

6. Mathematical Foundation

Usually classification:

$$P(y=c|x) = \text{softmax}(W \cdot h + b)_c$$

$$L = - \log P(y^* | x)$$

For rating regression:

$$\text{MSE} = (y - \hat{y})^2$$

Ordinal ratings may use ordinal classification instead of plain regression.

7. Practical Implementation

```
from transformers import pipeline
```

```
clf = pipeline("sentiment-analysis")

examples = [
    "The delivery was fast and the phone works perfectly.",
    "The screen is nice, but the battery life is terrible.",
]

for text in examples:
    print(text, "=>", clf(text)[0])
```

8. Code Explanation

- The pipeline loads a pretrained sentiment classifier.
- Each text receives a label and confidence score.
- Mixed-aspect text may be compressed into one label, which can hide nuance.

9. Training / Evaluation

- Datasets: IMDb, SST-2, Amazon reviews, Twitter sentiment datasets.
- Metrics: accuracy for balanced binary sentiment, macro F1 for imbalanced/multi-class.
- Improve using domain-specific training data, aspect labels, negation-aware preprocessing, and error analysis.

10. Complexity and Cost

- Lexicon and TF-IDF models are cheap.
- Transformer sentiment models are moderate cost and fast enough for APIs.
- Batch inference improves GPU utilization.

11. Common Use Cases

- Review mining, social listening, customer support prioritization, churn detection, market research.

12. Common Mistakes

- Treating star rating as perfect sentiment label.
- Ignoring neutral and mixed sentiment.
- Removing punctuation that signals emotion.
- Training on one domain and deploying to another.
- Using overall sentiment when aspect sentiment is required.

13. Edge Cases / Limitations

- Sarcasm, irony, slang, code-mixed language.
- Reviews with both praise and complaints.
- Domain-specific words: "sick" can be positive in slang.
- Fake reviews and bots.

14. Variations

- Lexicon-based sentiment: simple and interpretable.
- Binary sentiment: positive/negative.
- Multi-class sentiment: positive/neutral/negative.
- Emotion classification: anger, joy, sadness.
- Aspect-based sentiment: best for product analytics.

15. Related Topics

- Sentiment vs text classification: sentiment is a label type.
- Sentiment vs emotion: polarity vs emotional category.
- Aspect sentiment vs NER: aspect extraction plus opinion classification.

16. Interview Questions

1. What is sentiment analysis? Predicting opinion polarity or emotion from text.
2. Why is sarcasm hard? Literal words conflict with intended meaning.
3. What is aspect-based sentiment? Sentiment tied to a specific feature or entity.
4. Good metrics? Accuracy for balanced binary; macro F1 for multi-class/imbalance.
5. Why not remove stopwords blindly? Negation words are important.
6. How handle mixed sentiment? Use aspect-level labels or multi-label classification.
7. What is domain shift? Training and deployment text differ.
8. Can star ratings be noisy? Yes, text and rating may disagree.
9. Lexicon vs ML? Lexicon is transparent; ML learns context.
10. How improve production sentiment? Collect domain labels and monitor drift.

17. Practice Tasks

- Train IMDb sentiment classifier.
- Compare TF-IDF and BERT.
- Build aspect sentiment for restaurant reviews.
- Analyze sarcastic examples.
- Calibrate confidence thresholds.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Review Insight Dashboard	Summarizes product sentiment	sklearn/Streamlit	Amazon reviews	Business analytics
Tweet Emotion Monitor	Tracks emotions by topic	Transformers	GoEmotions	Social NLP
Aspect Review Miner	Extracts feature-level sentiment	spaCy/BERT	SemEval ABSA	Strong NLP project

19. Quick Revision

- Key idea: opinion detection.

- Formula: softmax cross-entropy.
- Metrics: accuracy, macro F1.
- Trap: mixed sentiment and sarcasm.
- Interview one-liner: Sentiment analysis is text classification focused on opinions.

20. Final Cheat Sheet

Item	Notes
Definition	Predicts sentiment/emotion
Input/output	Text -> polarity/emotion
Steps	Label, train, evaluate, inspect errors
Hyperparameters	model, max length, threshold
Pros	Valuable business signal
Cons	Sarcasm and domain shift

Question Answering

1. Overview

Question answering (QA) systems return answers to user questions. They may extract spans from context, generate answers from documents, answer multiple-choice questions, or combine retrieval with generation.

2. Intuition

QA is an open-book exam. Given a question and sometimes a passage, the model finds or writes the answer.

3. Prerequisites

- Tokenization and embeddings
- Reading comprehension datasets
- Span prediction
- Retrieval and ranking
- Exact match and F1

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Extractive QA	Selects answer span from context	More grounded	SQuAD	Start/end logits
Generative QA	Generates answer text	Flexible	LLM answer	Hallucination

Concept	Meaning	Why it matters	Example	Interview angle
Closed-book QA	Uses model memory only	No context required	trivia	Factual risk
Open-book QA	Uses retrieved documents	More current/grounded	RAG QA	Retrieval quality
Unanswerable QA	Detects no answer	Prevents false answers	SQuAD 2.0	Calibration

5. Algorithm / Working Process

Extractive QA:

1. Concatenate question and context.
2. Encode with transformer.
3. Predict start and end position logits.
4. Select best valid span.
5. Return text span with confidence.

Generative QA:

1. Retrieve or provide context.
2. Prompt model with question and context.
3. Generate answer.
4. Optionally cite sources and verify answer.

6. Mathematical Foundation

Extractive QA:

$$P_{\text{start}}(i) = \text{softmax}(s_i)$$

$$P_{\text{end}}(j) = \text{softmax}(e_j)$$

$$L = -\log P_{\text{start}}(i^*) - \log P_{\text{end}}(j^*)$$

Best span:

$$\text{argmax}_{\{i \leq j, j-i \leq \text{max_len}\}} P_{\text{start}}(i) P_{\text{end}}(j)$$

7. Practical Implementation

```
from transformers import pipeline

qa = pipeline("question-answering")

context = """
Transformers use self-attention to build contextual token representations.
They are widely used in NLP tasks such as translation and question answering.
"""
```

```
result = qa(  
    question="What mechanism do Transformers use?",  
    context=context,  
)  
  
print(result["answer"], round(result["score"], 3))
```

8. Code Explanation

- The pipeline receives a question and a context passage.
- The model predicts answer start and end positions.
- The output includes answer span, score, and offsets.

9. Training / Evaluation

- Datasets: SQuAD, Natural Questions, TriviaQA, HotpotQA.
- Metrics: exact match, token F1, answerability accuracy, citation correctness.
- Improve with better retrieval, domain fine-tuning, reranking, and no-answer threshold tuning.

10. Complexity and Cost

- Extractive QA is cheaper than generative QA.
- Long contexts require chunking and reranking.
- RAG QA cost includes embedding, vector search, prompt tokens, and generation.

11. Common Use Cases

- Document assistants, support bots, legal search, medical guidelines, enterprise knowledge bases, education tutors.

12. Common Mistakes

- Evaluating only answer text without checking source.
- Assuming retrieved context is relevant.
- Not handling unanswerable questions.
- Chunking documents poorly.
- Letting model answer outside evidence when grounding is required.

13. Edge Cases / Limitations

- Multi-hop reasoning across documents.
- Ambiguous questions.
- Contradictory sources.
- Tables, images, and scanned PDFs.
- Questions requiring current data.

14. Variations

- Extractive QA: reliable when answer is in text.
- Generative QA: flexible and natural.
- Multiple-choice QA: chooses from options.
- Conversational QA: uses chat history.
- RAG QA: combines retrieval and generation.

15. Related Topics

- QA vs RAG: QA is task; RAG is a common architecture.
- Extractive vs generative: span selection vs text generation.
- QA vs search: answer synthesis vs document ranking.

16. Interview Questions

1. What is extractive QA? Selecting an answer span from context.
2. How are start/end positions trained? Cross-entropy over token positions.
3. What is exact match? Predicted answer exactly equals gold answer after normalization.
4. Why is no-answer handling important? It prevents hallucinated spans.
5. What is open-book QA? QA using external documents.
6. How does RAG help QA? Retrieves evidence before generation.
7. Why can generative QA hallucinate? It may use learned priors beyond context.
8. How handle long documents? Chunk, retrieve, rerank, then answer.
9. What is multi-hop QA? Answer requires combining multiple facts.
10. How evaluate enterprise QA? Answer correctness, citation support, latency, and refusal quality.

17. Practice Tasks

- Run extractive QA on custom context.
- Build a PDF QA system with retrieval.
- Tune no-answer threshold.
- Compare extractive and generative QA.
- Evaluate citations manually.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
PDF QA Bot	Answers from uploaded PDFs	LangChain/HF/FastAPI	Company docs	Practical GenAI
SQuAD Trainer	Fine-tunes QA model	PyTorch/HF	SQuAD	Core NLP
Legal QA	Answers contract questions	RAG	CUAD	High-value domain

19. Quick Revision

- Key idea: answer questions from context or knowledge.

- Formula: start/end cross-entropy for extractive QA.
- Metrics: EM, F1, groundedness.
- Trap: hallucinating when answer is absent.
- Interview one-liner: QA turns documents or model knowledge into direct answers.

20. Final Cheat Sheet

Item	Notes
Definition	System that answers natural-language questions
Input/output	Question/context -> answer
Steps	Retrieve/read, predict/generate, verify
Hyperparameters	max span, top-k docs, threshold
Pros	Direct user value
Cons	Ambiguity, hallucination, retrieval dependence

Summarization

1. Overview

Summarization compresses long text into shorter text while preserving important information. It is used for news, meetings, legal documents, research papers, customer tickets, and enterprise knowledge workflows.

2. Intuition

Summarization is like writing lecture notes from a long chapter: keep the main ideas, remove repetition, and avoid changing facts.

3. Prerequisites

- Seq2seq models
- Attention and transformers
- ROUGE and human evaluation
- Faithfulness and factual consistency

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Extractive	Selects original sentences	More faithful	key sentence selection	Pros/cons
Abstractive	Generates new wording	More natural	paraphrased summary	Hallucination

Concept	Meaning	Why it matters	Example	Interview angle
Compression ratio	Output length/input length	Controls detail	10% summary	Length tuning
Faithfulness	Summary supported by source	Critical in production	no invented claim	Evaluation challenge
Coverage	Captures key points	Avoids missing facts	all decisions in meeting	ROUGE limitation

5. Algorithm / Working Process

1. Split long input into chunks if needed.
2. Encode source text.
3. Decode summary autoregressively.
4. Use beam search or sampling depending on use case.
5. Evaluate length, coverage, and faithfulness.
6. For very long documents, summarize chunks then combine summaries.

6. Mathematical Foundation

Abstractive summarization uses seq2seq likelihood:

$$P(s \mid d) = \prod_t P(s_t \mid s_{<t}, d)$$

Loss:

$$L = - \sum_t \log P(s_t^* \mid s_{<t}^*, d)$$

ROUGE recall:

$$\text{ROUGE-N} = \frac{\text{overlapping n-grams}}{\text{reference n-grams}}$$

7. Practical Implementation

```
from transformers import pipeline

summarizer = pipeline("summarization", model="facebook/bart-large-cnn")

article = """
Large language models are increasingly used in enterprise search, document
automation, and assistant workflows. Strong systems combine retrieval,
prompting, evaluation, and monitoring to reduce hallucination and improve
user trust.
"""

summary = summarizer(article, max_length=45, min_length=15, do_sample=False)
print(summary[0]["summary_text"])
```

8. Code Explanation

- BART is an encoder-decoder model trained for generation.
- `max_length` and `min_length` control summary size.
- `do_sample=False` makes output deterministic.

9. Training / Evaluation

- Datasets: CNN/DailyMail, XSum, SAMSum, arXiv/PubMed.
- Metrics: ROUGE, BERTScore, factual consistency checks, human evaluation.
- Improve with domain data, length control, retrieval, factuality reranking, and citation-aware prompting.

10. Complexity and Cost

- Long inputs are expensive due to attention cost.
- Generation cost scales with output length.
- Map-reduce summarization adds multiple model calls.

11. Common Use Cases

- Meeting notes, news briefs, support ticket summaries, paper summarization, legal document review.

12. Common Mistakes

- Trusting ROUGE alone.
- Summarizing chunks independently and losing global context.
- Allowing unsupported facts.
- Using too small max length.
- Not preserving dates, numbers, and names.

13. Edge Cases / Limitations

- Long documents with dispersed facts.
- Contradictory source sections.
- Tables and figures.
- Need for legal/medical precision.
- Highly technical content.

14. Variations

- Extractive summarization: safer and simpler.
- Abstractive summarization: fluent but may hallucinate.
- Query-focused summarization: summary answers a specific question.
- Multi-document summarization: combines sources.
- Hierarchical summarization: chunks then global summary.

15. Related Topics

- Summarization vs QA: broad compression vs question-specific answer.

- Extractive vs abstractive: selecting vs generating.
- RAG vs summarization: retrieval can select source context before summary.

16. Interview Questions

1. What is summarization? Condensing text while preserving key information.
2. Extractive vs abstractive? Select sentences vs generate new text.
3. Why can summaries hallucinate? Generation optimizes likelihood, not factual guarantee.
4. What is ROUGE? N-gram overlap metric.
5. Limitation of ROUGE? It misses factual correctness and paraphrases.
6. How summarize long docs? Chunk, summarize, merge, verify.
7. What is query-focused summary? Summary targeted to a user question.
8. How control length? max/min tokens, compression ratio, prompts.
9. How improve faithfulness? Cite sources, constrain generation, verify claims.
10. What is coverage? Capturing all important points.

17. Practice Tasks

- Summarize news with BART.
- Compare ROUGE for different max lengths.
- Build meeting-note summarizer.
- Detect hallucinated names/numbers.
- Implement extractive TextRank baseline.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Meeting Minutes Bot	Creates action-item summaries	Whisper, BART/LLM	AMI/SAMSum	End-to-end AI
Paper Summarizer	Summarizes abstracts and sections	Longformer/LLM	arXiv	Research friendly
Support Digest	Groups and summarizes tickets	RAG, Streamlit	support logs	Business value

19. Quick Revision

- Key idea: compress text.
- Formula: seq2seq negative log-likelihood.
- Metrics: ROUGE, BERTScore, faithfulness.
- Trap: fluent hallucinations.
- Interview one-liner: Summarization converts long text into faithful concise text.

20. Final Cheat Sheet

Item	Notes
Definition	Long text -> short text
Input/output	Document -> summary
Steps	chunk, encode, generate/select, verify
Hyperparameters	max length, beam width, chunk size
Pros	Saves reading time
Cons	May omit or invent facts

Machine Translation

1. Overview

Machine translation converts text from one language to another. It is a classic seq2seq task used in global products, customer support, localization, cross-lingual search, and multilingual assistants.

2. Intuition

The model learns meaning in the source language and expresses it naturally in the target language, preserving semantics rather than translating word by word.

3. Prerequisites

- Seq2seq models and attention
- Subword tokenization
- Parallel corpora
- BLEU/chrF/COMET
- Multilingual embeddings

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Parallel corpus	Source-target sentence pairs	Supervised training data	English-Hindi pairs	Data quality
Alignment	Source-target word relation	Helps understand attention	"house" -> "घर"	Attention is soft alignment
Subwords	Split rare words into pieces	Handles morphology/OOV	BPE tokens	Why BPE
Domain adaptation	Tune for domain language	Legal/medical terms	contract translation	General vs domain MT
Evaluation	Measures translation quality	Hard because many valid translations	BLEU	BLEU limitations

5. Algorithm / Working Process

1. Collect and clean parallel text.
2. Learn or use tokenizer.
3. Train encoder-decoder model to maximize target likelihood.
4. Decode using beam search.
5. Evaluate automatic metrics and human adequacy/fluency.
6. Deploy with terminology constraints if required.

6. Mathematical Foundation

Translation probability:

$$P(y_{\text{target}} \mid x_{\text{source}}) = \prod_t P(y_t \mid y_{<t}, x_{\text{source}})$$

Loss:

$$L = -\sum_t \log P(y_t^* \mid y_{<t}^*, x)$$

BLEU uses modified n-gram precision with brevity penalty:

$$\text{BLEU} = \text{BP} * \exp(\sum_n w_n \log p_n)$$

7. Practical Implementation

```
from transformers import MarianMTModel, MarianTokenizer

model_name = "Helsinki-NLP/opus-mt-en-fr"
tokenizer = MarianTokenizer.from_pretrained(model_name)
model = MarianMTModel.from_pretrained(model_name)

text = "Machine translation helps people communicate across languages."
inputs = tokenizer([text], return_tensors="pt", padding=True)
translated = model.generate(**inputs, num_beams=4, max_new_tokens=60)

print(tokenizer.decode(translated[0], skip_special_tokens=True))
```

8. Code Explanation

- MarianMT provides pretrained translation models.
- Tokenizer handles source language subwords.
- Beam search improves deterministic translation quality.
- Decoded tokens are converted back to target-language text.

9. Training / Evaluation

- Datasets: OPUS, WMT, IWSLT, FLORES.
- Metrics: BLEU, chrF, COMET, human adequacy and fluency.
- Improve with back-translation, domain fine-tuning, terminology constraints, and data filtering.

10. Complexity and Cost

- Similar to seq2seq transformer cost.
- Multilingual models are larger but share capacity.
- Production MT requires low latency and batching.

11. Common Use Cases

- Website localization, multilingual chat, document translation, cross-border commerce, travel apps.

12. Common Mistakes

- Word-by-word translation assumptions.
- Ignoring cultural and domain context.
- Evaluating only BLEU.
- Training on misaligned sentence pairs.
- Poor handling of names and numbers.

13. Edge Cases / Limitations

- Low-resource languages.
- Idioms and humor.
- Gender/formality ambiguity.
- Code-mixed text.
- Domain-specific terminology.

14. Variations

- Statistical MT: phrase tables, historical.
- Neural MT: modern seq2seq.
- Multilingual MT: one model for many languages.
- Zero-shot MT: translate unseen language pairs.
- Speech translation: audio to translated text.

15. Related Topics

- MT vs transliteration: meaning transfer vs script conversion.
- BLEU vs COMET: lexical overlap vs learned quality metric.
- Back-translation: monolingual target data creates synthetic pairs.

16. Interview Questions

1. What is machine translation? Converting text between languages.
2. Why use subword tokenization? Handles rare and morphologically rich words.
3. What is BLEU? N-gram precision metric with brevity penalty.
4. BLEU limitation? Penalizes valid paraphrases and misses meaning errors.
5. What is back-translation? Translate target monolingual text back to source to create training pairs.
6. Why beam search in MT? Finds better full-sentence translations.
7. What is domain adaptation? Fine-tuning MT for a specific domain.

8. What are low-resource challenges? Little parallel data and weak evaluation.
9. How handle terminology? Constrained decoding or glossary-aware fine-tuning.
10. What is adequacy vs fluency? Meaning preservation vs natural target language.

17. Practice Tasks

- Translate with MarianMT.
- Compute BLEU on sample pairs.
- Compare beam widths.
- Fine-tune on small domain corpus.
- Analyze idiom translation errors.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Domain Translator	Translates medical/legal text	MarianMT	OPUS/domain corpus	Applied NLP
BLEU Lab	Compares MT systems	sacreBLEU	WMT samples	Evaluation depth
Multilingual Chat Helper	Detects language and translates	fastText/HF	FLORES	Product-ready

19. Quick Revision

- Key idea: source language -> target language.
- Formula: conditional seq2seq likelihood.
- Metrics: BLEU, chrF, COMET.
- Trap: literal translation.
- Interview one-liner: MT is conditional generation preserving meaning across languages.

20. Final Cheat Sheet

Item	Notes
Definition	Automatic language translation
Input/output	Source sentence -> target sentence
Steps	tokenize, encode, decode, evaluate
Hyperparameters	beam width, max length, LR
Pros	Global communication
Cons	low-resource, idioms, bias

Retrieval-Augmented Generation

1. Overview

Retrieval-augmented generation (RAG) combines information retrieval with a generative model. Instead of relying only on model parameters, it retrieves relevant external documents and uses them as context for generation. It is common in enterprise chatbots, document QA, research assistants, and support automation.

2. Intuition

RAG is like giving an LLM an open notebook before asking it a question. The model answers using retrieved notes, reducing hallucination and allowing updates without retraining.

3. Prerequisites

- Embeddings and vector similarity
- Chunking and indexing
- Dense/sparse retrieval
- Prompting and LLM generation
- Evaluation of retrieval and answer quality

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Chunking	Split documents into passages	Retrieval unit quality	500-token chunks	Chunk size tradeoff
Embeddings	Dense vector representation	Semantic search	query vector	Cosine similarity
Vector DB	Stores searchable vectors	Fast retrieval	FAISS/Chroma	ANN search
Reranking	Reorders retrieved docs	Improves relevance	cross-encoder	Recall vs precision
Grounding	Answer supported by context	Reduces hallucination	cite passage	Faithfulness

5. Algorithm / Working Process

1. Ingest documents.
2. Split into chunks with metadata.
3. Embed chunks and store vectors.
4. Embed user query.
5. Retrieve top-k similar chunks.
6. Optionally rerank chunks.
7. Put context into prompt.
8. Generate answer with citations.
9. Evaluate retrieval and answer faithfulness.

6. Mathematical Foundation

Cosine similarity:

$$\cos(q, d) = (q \cdot d) / (||q|| ||d||)$$

Retriever objective often uses contrastive learning:

$$L = -\log \exp(\text{sim}(q, d) / \tau) / \sum_j \exp(\text{sim}(q, d_j) / \tau)$$

Answer generation:

$$P(y \mid q, D_k) = \prod_t P(y_t \mid y_{<t}, q, D_k)$$

7. Practical Implementation

```
from sentence_transformers import SentenceTransformer
import numpy as np

docs = [
    "RAG retrieves external documents before generation.",
    "Fine-tuning updates model weights using training examples.",
    "Beam search keeps multiple candidate generations.",
]

model = SentenceTransformer("all-MiniLM-L6-v2")
doc_emb = model.encode(docs, normalize_embeddings=True)

query = "How does retrieval augmented generation work?"
q_emb = model.encode([query], normalize_embeddings=True)[0]

scores = doc_emb @ q_emb
top = int(np.argmax(scores))

print("Retrieved:", docs[top])
print("Score:", round(float(scores[top]), 3))
```

8. Code Explanation

- Documents are embedded into dense vectors.
- Query is embedded in the same vector space.
- Dot product equals cosine similarity because embeddings are normalized.
- The top-scoring document is the retrieved context.

9. Training / Evaluation

- Retrieval metrics: recall@k, precision@k, MRR, nDCG.
- Generation metrics: groundedness, answer correctness, citation accuracy, latency.
- Improve with better chunking, hybrid search, metadata filters, reranking, query rewriting, and eval sets.

10. Complexity and Cost

- Indexing cost: embedding all chunks.
- Query cost: embedding query + vector search + LLM tokens.
- Memory: number of chunks times embedding dimension.
- ANN search makes large-scale retrieval practical.

11. Common Use Cases

- Enterprise knowledge assistants, PDF QA, support bots, policy search, codebase assistants, research tools.

12. Common Mistakes

- Bad chunking.
- No evaluation set.
- Retrieving irrelevant but semantically similar chunks.
- Stuffing too much context into prompt.
- No citations or source validation.
- Assuming RAG eliminates hallucination completely.

13. Edge Cases / Limitations

- Missing documents cannot be retrieved.
- Conflicting sources require resolution logic.
- Tables and images need special parsing.
- Queries with vague references need conversation context.
- Private data requires access control.

14. Variations

- Sparse retrieval: BM25, strong lexical baseline.
- Dense retrieval: semantic search with embeddings.
- Hybrid retrieval: combines sparse and dense.
- Agentic RAG: model plans multiple retrieval steps.
- Graph RAG: uses entity/relation structure.

15. Related Topics

- RAG vs fine-tuning: RAG updates knowledge via documents; fine-tuning changes behavior/weights.
- RAG vs search: RAG synthesizes answers from retrieved content.
- Reranking vs retrieval: first gets candidates, second improves ordering.

16. Interview Questions

1. What is RAG? Retrieval plus generation using external context.
2. Why use RAG? Fresh knowledge, grounding, and lower retraining cost.
3. What is chunking? Splitting documents into retrievable passages.
4. What is cosine similarity? Normalized vector similarity.
5. What is recall@k? Fraction of questions where relevant doc appears in top k.
6. BM25 vs dense retrieval? Lexical matching vs semantic similarity.

7. Why rerank? Initial retrieval may miss fine-grained relevance.
8. Does RAG remove hallucinations? No, it reduces them when retrieval and prompting are good.
9. How handle access control? Filter documents before retrieval/generation.
10. How evaluate RAG? Retrieval quality, answer correctness, groundedness, latency.

17. Practice Tasks

- Build small vector search over notes.
- Add metadata filtering.
- Compare BM25 and embeddings.
- Create 20 QA eval examples.
- Add citations to generated answers.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Company Docs Chatbot	Answers from internal docs	FAISS, FastAPI, LLM	docs/PDFs	AI engineer role
Research Paper Assistant	Retrieves paper sections	SentenceTransformers	arXiv papers	Research value
Policy QA	Answers HR/legal policy questions	Hybrid search	policy docs	Enterprise-ready

19. Quick Revision

- Key idea: retrieve evidence, then generate.
- Formula: cosine similarity and conditional generation.
- Metrics: recall@k, groundedness, answer correctness.
- Trap: poor chunking.
- Interview one-liner: RAG gives LLMs external memory at inference time.

20. Final Cheat Sheet

Item	Notes
Definition	Generation conditioned on retrieved documents
Input/output	Query/docs -> grounded answer
Steps	chunk, embed, retrieve, rerank, prompt, answer
Hyperparameters	chunk size, top-k, embedding model
Pros	Fresh, grounded, auditable
Cons	retrieval failures, latency, context limits

Instruction Tuning

1. Overview

Instruction tuning fine-tunes a pretrained language model on datasets of instructions and desired responses. It teaches the model to follow natural-language tasks, making it more useful as an assistant.

2. Intuition

Pretraining teaches language patterns. Instruction tuning teaches manners and task following: when a user asks "summarize this", the model learns to produce a summary instead of merely continuing text.

3. Prerequisites

- Language modeling
- Supervised fine-tuning
- Prompt-response datasets
- Cross-entropy loss
- Evaluation of helpfulness and task accuracy

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
SFT	Supervised fine-tuning	Main training method	instruction -> answer	Pretraining vs SFT
Instruction dataset	Task prompts and outputs	Teaches following behavior	Alpaca-style data	Data quality
Chat template	Formats roles/messages	Must match inference format	system/user/assistant	Tokenization bugs
Response masking	Loss only on assistant answer	Avoids training on prompt	ignore user tokens	Common implementation
Generalization	Follow unseen instructions	Main goal	new task	Benchmarking

5. Algorithm / Working Process

1. Start with pretrained LM.
2. Prepare instruction-response examples.
3. Format using chat template.
4. Mask prompt tokens if training only on assistant response.
5. Optimize next-token cross-entropy.
6. Evaluate on held-out tasks and human preference.

6. Mathematical Foundation

For prompt x and target response y :

$$P(y|x) = \prod_t P(y_t \mid x, y_{<t})$$

SFT loss:

$$L_{\text{SFT}} = -\sum_t m_t \log P(y_t \mid x, y_{<t})$$

where $m_t=1$ for response tokens and 0 for ignored prompt tokens.

7. Practical Implementation

```
from datasets import Dataset
from transformers import AutoTokenizer

rows = [
    {
        "instruction": "Explain overfitting in one sentence.",
        "response": "Overfitting happens when a model memorizes training data and performs poorly on unseen data.",
    }
]

ds = Dataset.from_list(rows)
tokenizer = AutoTokenizer.from_pretrained("distilgpt2")
tokenizer.pad_token = tokenizer.eos_token

def format_example(row):
    text = f"### Instruction:\n{row['instruction']}\n\n### Response:\n{row['response']}"
    return tokenizer(text, truncation=True, padding="max_length", max_length=128)

tokenized = ds.map(format_example)
print(tokenized[0]["input_ids"][:10])
```

8. Code Explanation

- A tiny instruction dataset is created.
- The example is formatted into prompt-response text.
- Tokenization converts it to training-ready IDs.
- Real SFT would pass these IDs to a causal LM trainer with labels.

9. Training / Evaluation

- Use diverse, high-quality instructions.
- Evaluate on unseen tasks, safety cases, factual QA, and formatting compliance.
- Watch for overfitting, response style collapse, and benchmark contamination.
- Hyperparameters: learning rate, batch size, sequence length, epochs, LoRA rank.

10. Complexity and Cost

- Cheaper than pretraining, more expensive than prompting.

- LoRA/QLoRA can fine-tune on modest GPUs.
- Data quality often matters more than dataset size.

11. Common Use Cases

- Chat assistants, coding assistants, domain support bots, structured extraction, enterprise assistants.

12. Common Mistakes

- Low-quality synthetic instructions.
- Mismatched train and inference templates.
- Training on user prompt tokens unintentionally.
- Too many epochs causing memorization.
- Not evaluating instruction-following behavior directly.

13. Edge Cases / Limitations

- Does not reliably add factual knowledge.
- Can learn bad style or unsafe behavior from data.
- Hard to evaluate broad assistant quality.
- May reduce base-model capabilities if overdone.

14. Variations

- Full fine-tuning: updates all weights.
- LoRA: trains low-rank adapters.
- QLoRA: quantized base model plus LoRA.
- Multi-task instruction tuning: many task families.
- Self-instruct: synthetic instruction generation.

15. Related Topics

- Instruction tuning vs RLHF: supervised imitation vs preference alignment.
- Fine-tuning vs RAG: behavior adaptation vs external knowledge.
- LoRA vs full fine-tuning: efficient adapters vs full weight updates.

16. Interview Questions

1. What is instruction tuning? SFT on instruction-response pairs.
2. Why do it after pretraining? To teach task-following behavior.
3. What loss is used? Next-token cross-entropy on target response.
4. What is response masking? Ignoring prompt tokens in loss.
5. Does instruction tuning add knowledge? Not reliably; it mainly changes behavior.
6. What is a chat template? Role-based formatting used during training/inference.
7. Why can template mismatch hurt? Model sees different token patterns at inference.
8. How evaluate instruction tuning? Held-out tasks, human eval, safety, formatting.
9. LoRA advantage? Lower memory and faster fine-tuning.
10. Risk of bad SFT data? The model imitates poor answers.

17. Practice Tasks

- Format a small instruction dataset.
- Fine-tune a small causal LM with LoRA.
- Compare base vs instruction-tuned outputs.
- Evaluate JSON-format following.
- Analyze failures from prompt-template mismatch.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Domain Tutor LM	Answers course questions	LoRA, HF	custom Q/A	Fine-tuning proof
JSON Extractor Model	Outputs structured fields	SFT	synthetic extraction data	AI engineering
Interview Coach Bot	Gives placement answers	QLoRA	curated prompts	Direct placement value

19. Quick Revision

- Key idea: teach pretrained LM to follow instructions.
- Formula: masked next-token cross-entropy.
- Metrics: task success, helpfulness, format accuracy.
- Trap: poor data/template mismatch.
- Interview one-liner: Instruction tuning converts a language model into a task-following assistant.

20. Final Cheat Sheet

Item	Notes
Definition	SFT on instruction-response pairs
Input/output	Instruction -> desired response
Steps	format, tokenize, mask, fine-tune
Hyperparameters	LR, epochs, sequence length, LoRA rank
Pros	Better controllability
Cons	Data-sensitive, not a knowledge solution

RLHF

1. Overview

Reinforcement Learning from Human Feedback (RLHF) aligns language models with human preferences. It usually trains a reward model from human comparisons and then optimizes the policy model to produce preferred responses.

2. Intuition

Instead of telling the model only the correct answer, humans compare two answers and say which is better. The model learns a reward signal from those preferences, then improves toward higher-reward responses.

3. Prerequisites

- Supervised fine-tuning
- Reinforcement learning basics
- Reward models
- KL divergence
- Policy optimization, especially PPO

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Preference data	Human choice between outputs	Easier than writing ideal answers	A better than B	Pairwise loss
Reward model	Scores responses	Converts preference to scalar reward	$r(x, y)$	Reward hacking
Policy model	LLM being optimized	Generates improved answers	assistant model	PPO
KL penalty	Keeps model near reference	Prevents drift	penalty vs SFT model	Alignment stability
PPO	RL optimizer	Common RLHF algorithm	clipped objective	Why not pure reward max

5. Algorithm / Working Process

1. Pretrain language model.
2. Perform supervised instruction tuning.
3. Generate multiple responses for prompts.
4. Humans rank or compare responses.
5. Train reward model to predict preferences.
6. Optimize policy using PPO with KL penalty.
7. Evaluate helpfulness, harmlessness, honesty, and regressions.

6. Mathematical Foundation

Reward model pairwise loss:

$$L = -\log \text{sigma}(r(x, y_w) - r(x, y_l))$$

where y_w is preferred over y_l .

RLHF objective:

$$\text{maximize } E[r(x,y)] - \beta \text{KL}(\pi_{\theta}(.|x) || \pi_{\text{ref}}(.|x))$$

PPO uses clipped policy ratio:

$$\min(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) A_t)$$

7. Practical Implementation

```
import torch
import torch.nn.functional as F

reward_win = torch.tensor([2.3, 1.2, 0.7])
reward_lose = torch.tensor([0.4, 0.8, 1.1])

loss = -F.logsigmoid(reward_win - reward_lose).mean()
print(float(loss))
```

8. Code Explanation

- The winning response should receive a higher reward than the losing response.
- $\text{reward_win} - \text{reward_lose}$ is the preference margin.
- `logsigmoid` implements the Bradley-Terry style pairwise preference loss.

9. Training / Evaluation

- Data: human pairwise comparisons or rankings.
- Metrics: win rate against baseline, helpfulness, safety, refusal quality, reward-model accuracy.
- Watch for reward hacking and capability regressions.
- Human evaluation remains important.

10. Complexity and Cost

- Expensive due to human labeling, reward-model training, and RL optimization.
- PPO requires repeated generation and scoring.
- More operationally complex than SFT or DPO.

11. Common Use Cases

- Chat assistant alignment, safety tuning, style preference tuning, summarization preference optimization.

12. Common Mistakes

- Treating reward model score as ground truth.
- Ignoring KL penalty.
- Optimizing until outputs exploit reward model.

- Poor preference-labeling guidelines.
- Evaluating only with the reward model.

13. Edge Cases / Limitations

- Human preferences are inconsistent.
- Reward model can be gamed.
- Expensive and hard to reproduce.
- May over-align and reduce creativity/helpfulness.

14. Variations

- RLHF with PPO: classic pipeline.
- RLAIIF: AI feedback instead of human feedback.
- Constitutional AI: preference/safety principles guide feedback.
- DPO/IPO/KTO: preference optimization without explicit RL loop.

15. Related Topics

- RLHF vs instruction tuning: preference optimization vs supervised imitation.
- RLHF vs DPO: reward/PPO pipeline vs direct preference loss.
- Reward hacking in RL: optimizing proxy reward badly.

16. Interview Questions

1. What is RLHF? Optimizing models using human preference feedback.
2. Why use preference data? Humans can compare answers more easily than write perfect ones.
3. What is reward model? Model predicting scalar preference reward.
4. What is KL penalty? Constraint keeping policy close to reference model.
5. Why PPO? Stable policy optimization with clipped updates.
6. What is reward hacking? Model exploits reward model weaknesses.
7. RLHF vs SFT? SFT imitates examples; RLHF optimizes preferences.
8. What data is needed? Prompts, model responses, human comparisons.
9. Main evaluation? Human win rate and safety/capability tests.
10. Why is RLHF expensive? Human labels plus RL generation loops.

17. Practice Tasks

- Implement pairwise reward loss.
- Create a tiny preference dataset.
- Compare SFT response and preferred response.
- Study reward hacking examples.
- Read PPO objective and explain each term.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Reward Model Trainer	Learns pairwise preferences	PyTorch/HF	Anthropic HH subset	Alignment knowledge
Summary Preference Ranker	Scores better summaries	Transformers	preference pairs	Applied RLHF
Mini RLHF Simulator	Toy PPO over text choices	PyTorch	synthetic prompts	Research internship depth

19. Quick Revision

- Key idea: train reward from preferences, optimize model toward reward.
- Formula: $-\log \text{sigmoid}(r_w - r_l)$.
- Metrics: win rate, safety, reward accuracy.
- Trap: reward hacking.
- Interview one-liner: RLHF aligns LLM outputs with human preferences using a learned reward signal.

20. Final Cheat Sheet

Item	Notes
Definition	RL optimization from human preferences
Input/output	prompt/responses/preferences -> aligned policy
Steps	SFT, preference data, reward model, PPO
Hyperparameters	KL beta, PPO clip, LR
Pros	Aligns subjective quality
Cons	Expensive, unstable, reward hacking

Preference Optimization

1. Overview

Preference optimization trains models from preferred and rejected responses. Modern methods such as Direct Preference Optimization (DPO) avoid explicit reward-model training and PPO, making alignment simpler than classic RLHF.

2. Intuition

Given two answers, one preferred and one rejected, teach the model to increase the probability of the preferred answer relative to the rejected one, while staying close to the reference model.

3. Prerequisites

- Language-model log probabilities

- Preference datasets
- KL regularization intuition
- SFT and RLHF basics

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Chosen/rejected	Preferred pair labels	Training signal	answer A > B	Pairwise setup
Reference model	Usually SFT model	Stabilizes optimization	frozen copy	Why needed
Log-prob ratio	Preference evidence	Directly optimizes policy	$\log p(\text{chosen}) - \log p(\text{rejected})$	DPO formula
Beta	Strength of preference update	Controls deviation	$\text{beta}=0.1$	Stability
Offline training	No RL rollout loop	Simpler than PPO	fixed dataset	DPO vs RLHF

5. Algorithm / Working Process

1. Start with SFT model and frozen reference model.
2. Prepare prompt, chosen response, rejected response triples.
3. Compute log probability of chosen and rejected responses under both models.
4. Optimize model so chosen response becomes relatively more likely.
5. Evaluate win rate and regressions.

6. Mathematical Foundation

DPO loss:

$$\mathcal{L}_{\text{DPO}} = -\log \text{sigmoid}(\text{beta}[(\log \pi_{\theta}(y_w|x) - \log \pi_{\theta}(y_l|x)) - (\log \pi_{\text{ref}}(y_w|x) - \log \pi_{\text{ref}}(y_l|x))])$$

where y_w is chosen and y_l is rejected.

7. Practical Implementation

```
import torch
import torch.nn.functional as F

beta = 0.1
policy_chosen = torch.tensor([-12.0, -8.0])
policy_rejected = torch.tensor([-14.0, -7.5])
ref_chosen = torch.tensor([-11.5, -8.2])
ref_rejected = torch.tensor([-13.0, -7.8])
```

```
policy_margin = policy_chosen - policy_rejected
ref_margin = ref_chosen - ref_rejected
loss = -F.logsigmoid(beta * (policy_margin - ref_margin)).mean()

print(float(loss))
```

8. Code Explanation

- Log probabilities are sequence log-likelihoods.
- `policy_margin` measures how much the trainable model prefers chosen over rejected.
- `ref_margin` subtracts the reference model's existing preference.
- DPO increases preference beyond the reference baseline.

9. Training / Evaluation

- Use high-quality preference pairs.
- Metrics: pairwise accuracy, human/LLM win rate, safety, task performance.
- Hyperparameters: beta, learning rate, batch size, max length.
- Improve with better negative responses, data filtering, and balanced preference categories.

10. Complexity and Cost

- Cheaper and simpler than PPO-based RLHF.
- Requires computing sequence log probabilities for two responses and often a reference model.
- Can be trained with LoRA/QLoRA.

11. Common Use Cases

- Chatbot alignment, style tuning, safety behavior, concise answering, code assistant preference tuning.

12. Common Mistakes

- Using weak or ambiguous rejected responses.
- Forgetting reference-model term.
- Setting beta too high.
- Training on preference data that conflicts with desired policy.
- Measuring only training pair accuracy.

13. Edge Cases / Limitations

- Depends heavily on pair quality.
- Offline preferences may not cover new model behaviors.
- Can overfit style preferences.
- Does not guarantee factual correctness.

14. Variations

- DPO: most common direct method.
- IPO: alternative preference objective.

- KTO: uses desirable/undesirable examples.
- ORPO: combines SFT and preference signal.
- SimPO: simplified preference objective.

15. Related Topics

- Preference optimization vs RLHF: direct offline loss vs reward/PPO.
- SFT vs DPO: imitation vs relative preference.
- Ranking losses in recommender systems: similar pairwise idea.

16. Interview Questions

1. What is preference optimization? Training from chosen/rejected response pairs.
2. What is DPO? Directly optimizes preference likelihood without PPO.
3. Why use a reference model? To constrain deviation from original behavior.
4. What does beta control? Strength of preference optimization.
5. DPO vs RLHF? DPO avoids reward model and RL rollout.
6. What are chosen and rejected responses? Preferred and dispreferred outputs for same prompt.
7. What is sequence log probability? Sum of token log probabilities.
8. Can DPO improve factuality? Only if preferences encode factuality well.
9. Main risk? Overfitting noisy preferences.
10. How evaluate? Win rate, safety, capability benchmarks, human review.

17. Practice Tasks

- Compute DPO loss by hand for two pairs.
- Build a small chosen/rejected dataset.
- Train LoRA with a DPO trainer.
- Compare SFT and DPO outputs.
- Analyze effect of beta.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Concise Answer Tuner	Prefers shorter correct answers	TRL, LoRA	custom pairs	Alignment skill
Code Review Preference Model	Prefers actionable reviews	HF	synthetic/review pairs	Coding assistant angle
Safety Style Optimizer	Chooses safer responses	DPO	HH-RLHF	Research relevance

19. Quick Revision

- Key idea: make chosen responses more likely than rejected ones.
- Formula: DPO pairwise log-sigmoid loss.

- Metrics: win rate and regressions.
- Trap: noisy preference data.
- Interview one-liner: Preference optimization aligns models directly from pairwise choices.

20. Final Cheat Sheet

Item	Notes
Definition	Direct training from response preferences
Input/output	prompt + chosen/rejected -> tuned model
Steps	logprobs, margins, DPO loss
Hyperparameters	beta, LR, max length
Pros	Simpler than RLHF
Cons	Data quality sensitive

Tool-Using Agents

1. Overview

Tool-using agents are LLM systems that can call external tools such as search, calculators, databases, code interpreters, APIs, browsers, and workflow systems. They are used in AI copilots, coding agents, data-analysis assistants, customer support automation, and enterprise operations.

2. Intuition

An LLM alone is a brain with memory limits. Tools give it hands: it can look things up, calculate exactly, run code, fetch files, and take actions.

3. Prerequisites

- Prompting and function calling
- JSON schemas and APIs
- RAG basics
- Planning and control loops
- Safety, permissions, and evaluation

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Tool schema	Defines callable function	Makes calls structured	<code>search(query)</code>	JSON/function calling
Planner	Decides next action	Handles multi-step tasks	retrieve then summarize	ReAct

Concept	Meaning	Why it matters	Example	Interview angle
Observation	Tool result returned to model	Grounds next step	search result	Loop design
Guardrails	Limits unsafe actions	Prevents damage	approval before delete	Safety
State	Memory of task progress	Enables long workflows	files changed	Agent reliability

5. Algorithm / Working Process

1. Receive user goal.
2. Decide whether a tool is needed.
3. Select tool and construct valid arguments.
4. Execute tool.
5. Observe result.
6. Continue reasoning or call another tool.
7. Produce final answer or completed action.
8. Log actions and enforce permissions for risky tools.

6. Mathematical Foundation

An agent policy can be viewed as:

$$a_t \sim \pi(a_t \mid \text{user_goal}, \text{history}, \text{observations})$$

Tool-augmented generation:

$$P(\text{response} \mid x, o_1, \dots, o_T)$$

For evaluation:

$$\text{Task success rate} = \text{completed tasks} / \text{total tasks}$$
$$\text{Tool precision} = \text{useful tool calls} / \text{total tool calls}$$

7. Practical Implementation

```
import math

def calculator(expression: str) -> float:
    allowed = {"sqrt": math.sqrt, "log": math.log, "pi": math.pi}
    return eval(expression, {"__builtins__": {}}, allowed)

def simple_agent(question: str) -> str:
    if "sqrt" in question:
        expr = question.split("of", 1)[1].strip(" ?")
        value = calculator(f"sqrt({expr})")
        return f"The answer is {value:.3f}."
```

```
    return "I can answer directly only when no tool is needed."

print(simple_agent("What is the sqrt of 144?"))
```

8. Code Explanation

- `calculator` is a tiny restricted tool.
- `simple_agent` chooses the tool based on the question.
- Real agents use model-based tool selection and strict schemas instead of string heuristics.
- The restricted `eval` removes builtins and exposes only approved math functions.

9. Training / Evaluation

- Evaluate task success, tool-call correctness, latency, cost, and safety.
- Use benchmark tasks with expected tool traces where possible.
- Improve with better tool descriptions, schemas, examples, planning limits, retries, and verification steps.

10. Complexity and Cost

- Cost depends on number of model calls and tool calls.
- Latency can be high for multi-step loops.
- Reliability drops when tools are flaky or observations are long/noisy.
- Stateful agents require logs and recovery.

11. Common Use Cases

- Coding agents, data analysis, web research, calendar/email assistants, database copilots, RAG workflows, IT automation.

12. Common Mistakes

- Letting agents call dangerous tools without approval.
- Poor tool schemas.
- No maximum iteration limit.
- Not validating tool outputs.
- Using agents when a simple function or workflow is enough.

13. Edge Cases / Limitations

- Tool errors and partial failures.
- Prompt injection through retrieved content.
- Ambiguous user intent.
- Long-horizon task drift.
- Hidden side effects in external systems.

14. Variations

- ReAct agents: reason and act iteratively.
- Plan-and-execute agents: create plan then perform steps.

- Router agents: choose specialized tools/models.
- Multi-agent systems: split work among agents.
- Workflow agents: deterministic graph with LLM steps.

15. Related Topics

- Agents vs chains: dynamic decisions vs fixed sequence.
- Tools vs RAG: tools can include retrieval, calculators, APIs, code.
- Function calling vs free-form prompting: structured calls vs natural text.

16. Interview Questions

1. What is a tool-using agent? An LLM system that can call external functions/APIs.
2. Why use tools? To access fresh data, exact computation, and external actions.
3. What is a tool schema? Structured definition of inputs and outputs.
4. What is ReAct? A loop combining reasoning, actions, and observations.
5. Main safety issue? Tools can cause real-world side effects.
6. How prevent infinite loops? Step limits and stopping criteria.
7. How evaluate agents? Task success, tool precision, cost, latency, safety.
8. What is prompt injection? Malicious content tries to control the model through context.
9. Agents vs RAG? RAG retrieves context; agents choose actions/tools dynamically.
10. When not use an agent? When a deterministic function or simple pipeline solves the task.

17. Practice Tasks

- Build calculator tool calling.
- Add a search tool mock.
- Implement max-step loop.
- Log tool calls and observations.
- Test prompt-injection examples.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Data Analyst Agent	Runs SQL/Python and explains charts	Python, DuckDB, LLM	CSV datasets	AI engineer portfolio
Coding Assistant	Edits files and runs tests	Python, tools, git	local repos	Strong practical signal
Support Workflow Agent	Looks up policy and files ticket	RAG, APIs	support docs	Enterprise automation

19. Quick Revision

- Key idea: LLM chooses and uses external tools.
- Formula: policy over actions conditioned on history.
- Metrics: task success, tool precision, cost, safety.

- Trap: overusing agents for deterministic tasks.
- Interview one-liner: Tool-using agents extend LLMs from text generation to grounded action.

20. Final Cheat Sheet

Item	Notes
Definition	LLM system that calls external tools
Input/output	user goal -> actions + final result
Steps	decide, call, observe, repeat, answer
Hyperparameters	max steps, tool set, temperature
Pros	Fresh data, exact computation, action
Cons	safety, latency, reliability